

VLSI / Digital Design Verification Project

Interview Preparation Guide

Table of Contents

Table of Contents	2
1. Project at a Glance	7
Final project title	7
30-second explanation	7
What problem the project demonstrates	7
Why there are two DUTs	7
Overall verification architecture (one line)	7
What makes the project technically strong	8
What the project does NOT claim	8
2. Architecture -- How Everything Fits Together	9
Repository layout	9
UART environment (phase1_uart/uart_env/)	9
APB environment (phase5_apb/apb_env/)	9
Mutation engine (mutation/)	10
Unified regression framework (regression/, run_regression.py)	10
Formal verification layer (phase5_apb/formal/)	10
Lint	10
CI	10
phase2_llm/ (separate research track)	10
Simple architecture diagram	10
3. The Core Verification Story	12
4. Exact Measured Results	14
Test pass rate: 24/24 across 9 suites	14
Mutation score (kept separate per DUT and per mechanism)	14
Functional coverage	14
Formal: 3/3 properties PASS	14
Lint findings (not fixed -- documented trade-off)	15
Seed reproducibility	15
CI status	15
5. 5-Minute Recruiter Demonstration -- Step by Step	16

Exact commands used above	17
6. Live Demo Backup Plan	18
7. Best Mutation Demo	19
Original RTL (apb_regblock.v, CTRL write handling)	19
Mutated RTL (apb_regblock_mutant3.v)	19
What defect was injected	19
Which mechanism catches it	19
What the output looks like	19
The real story: it initially survived EVERYTHING	20
Exactly how to explain why mutation testing matters (script)	20
Why 9/9 does NOT mean the design is exhaustively verified	20
8. Waveform Demo	21
Exact steps	21
APB signal reference	21
SETUP vs. ACCESS phase	21
What to point out in the waveform (script)	21
9. Formal Verification -- Interview Preparation	22
Simple explanation first	22
Technically precise explanation	22
P1, P2, P3 -- exact status	22
Why P1 is bounded BUT complete for its specific claim	22
Why P2/P3 are unbounded	22
Assumptions used by the proofs (all explicit, none hidden)	22
What formal proves / does not prove	23
Toolchain limitations, verified not assumed	23
10. Mutation Testing -- Interview Preparation	24
What it is	24
Why it is useful	24
What 9/9 means (and the limits of that claim)	24
Per-mechanism attribution -- why it's the informative number	24
Equivalent mutants -- what they are and why they were excluded	24
Examples of defects caught by different mechanisms	24

11. Coverage -- Interview Preparation	26
Functional coverage	26
Coverpoints and bins	26
Constrained-random coverage	26
What 100% means in THIS project	26
What it does NOT mean	26
Why the coverage numbers are not an exhaustive state-space claim	26
12. Constrained-Random Testing	27
How stimulus is generated	27
How seeds work	27
Reproducibility	27
Corner-case injection	27
Why random testing is useful	27
How to reproduce a failure using a seed	27
13. Scoreboard / Reference Model	28
Why the reference model is independent	28
What the scoreboard compares	28
Why the scoreboard does not own latency checking (APB)	28
Why separation of concerns matters	28
14. Protocol Checker vs. Scoreboard vs. Latency Checker	29
15. UART -- What I Should Know	30
16. APB -- What I Should Know	31
Signals	31
SETUP / ACCESS	31
Registers	31
Invalid addresses / PSLVERR	31
START / BUSY / DONE / operation latency	31
Likely interview questions and concise answers	31
17. The LLM Experiment	33
What phase2_llm does	33
How the LLM-generated testbench was produced	33

v1 result: 5/6	33
What bug it missed, and why	33
v2 result: 6/6	33
What changed in the prompt / test	33
Why 6/6 is NOT comparable to the current 9/9	33
Why it remains a side experiment	34
What I should say in an interview	34
What I must NOT claim	34
18. 60-90 Second Project Explanation	35
19. 30-Second Version	36
20. "Tell Me About Your Project" Answer	37
21. Top Technical Interview Questions	38
10 hardest questions	38
Likely follow-up questions by topic	39
22. The Biggest Weaknesses / How to Answer Them	40
23. Career / Resume Section	41
3 strong project title options	41
5 detailed resume bullets	41
2-line resume version	41
ATS-friendly version (keyword-dense)	41
Version for a VLSI verification internship application	41
Version for a general ECE/VLSI fresher resume	42
24. LinkedIn / GitHub Project Description	43
Short GitHub description	43
Longer GitHub README-style description	43
LinkedIn project description	43
1-line portfolio description	43
25. Recruiter Demo Cheat Sheet	44
Commands	44
Files to open	44

Results to show	44
5 key things to say	44
5 things NOT to claim	44

26. Interview Cheat Sheet

45

Architecture, one line	45
5 most important numbers	45
5 strongest technical points	45
5 weakest points (own them, then pivot)	45
10 keywords/concepts I must understand	45
10 questions I should be able to answer cold	46

27. "If They Ask Me to Go Deeper"

47

28. Final Pre-Interview Checklist

49

1. Project at a Glance

Final project title

cocotb-Based Digital Design Verification Portfolio: UART Transmitter + APB4-Lite Register Block -- two independently-verified RTL designs, a shared DUT-agnostic mutation-testing engine, a unified reproducible regression framework with CI, and a small formal-verification layer.

30-second explanation

I built complete, professional-style verification environments for two structurally different digital designs -- a UART transmitter and an APB4-Lite register block -- each with its own driver, monitor, reference model, scoreboard, protocol checker, functional coverage, and constrained-random stimulus. On top of that I built a DUT-agnostic mutation-testing engine to measure whether my own testbenches actually catch bugs, a formal-verification layer that proves two properties true for all time on the register block, and a one-command regression framework wired into real GitHub Actions CI. Everything is free/open-source tooling -- no commercial EDA licenses.

What problem the project demonstrates

Not "can I write a testbench" -- almost any junior engineer can do that for a simple DUT. The actual question this project answers is: **how do you know your verification environment itself is trustworthy?** Every mechanism here (scoreboard, checker, coverage, mutation testing, formal) is deliberately kept separate, its exact scope documented, and its claims measured rather than assumed -- see `LESSONS_LEARNED.md` for real bugs found this way during development itself.

Why there are two DUTs

- A single DUT risks building an architecture that only works for that DUT's specific shape -- adding a second, structurally different DUT and reusing the pattern (not the code) proves the approach generalizes.
- The **mutation engine** (`mutation/framework.py`) and the **regression framework** (`regression/`) are literally the same, unmodified code driving both DUTs -- concrete proof of reuse, not just a claim.
- UART and APB deliberately share almost no other code: UART's driver is **non-blocking** (async open-loop frame), APB's driver is **blocking** (SETUP->ACCESS request/response). Forcing a shared base class between them would have been the kind of premature abstraction this project explicitly avoids.

DUT	What it is	Protocol shape	What's functionally interesting about it
UART TX	Parameterized 8N1 UART transmitter (<code>uart_tx.v</code>), configurable <code>CLKS_PER_BIT</code>	Async, open-loop frame (start bit, 8 data bits, stop bit)	Exact bit timing, LSB-first data order, busy-flag behavior, reset-mid-frame recovery
APB4-Lite regblock	CTRL/STATUS/DATA/INTCLR register block (<code>apb_regblock.v</code>), genuine multi-cycle operation	Blocking request/response (APB SETUP -> ACCESS)	Register-map legality (PSLVERR), START/BUSY/DONE control-flow guard, exact <code>OP_LATENCY</code> , illegal-address handling

Overall verification architecture (one line)

stimulus (driver + constrained-random sequences) → passive monitor → independent reference model → scoreboard (data-value judgement) + protocol/latency checkers (structural/timing judgement, independent of data) → functional coverage (did we exercise the right space) → mutation testing (does the environment actually detect bugs) → formal verification (exhaustive proof for a few chosen properties) → unified regression + CI (repeatable, automated, one command).

What makes the project technically strong

- **Separation of concerns is real, not cosmetic:** the scoreboard never checks timing, the protocol checker never knows the expected data value, the latency checker is fully independent of both -- each catches a genuinely different defect class (see Section 14).
- **Every non-trivial timing/behavioral claim was verified empirically,** not hand-derived -- seven-plus real bugs found this way are documented with root cause in `LESSONS_LEARNED.md`.
- **Mutation testing found and fixed two genuine gaps** (not a vanity 100% -- see Section 7) before reaching its final 9/9 score on each DUT.
- **Formal proofs are genuinely unbounded where claimed:** two of three APB properties pass by k-induction (true for all time), and the one bounded-BMC property is proven complete for its specific combinational claim, with the distinction explained rather than glossed over.
- **Reproducibility is verified, not assumed:** the same random seed reproduces byte-for-byte identical stimulus, confirmed directly, not just logged.
- **Real CI:** the regression actually runs green on GitHub Actions' free ubuntu-latest runners, not just locally.

What the project does NOT claim

IMPORTANT: This project is not "fully verified" or "100% verified." Every mechanism has a specific, bounded scope, documented in `VERIFICATION_STRATEGY.md`. Specifically, it does **not** claim: SystemVerilog Assertions or UVM/pyuvvm (procedural cocotb checkers only -- Icarus Verilog 12.0 was directly confirmed not to parse SVA property/sequence blocks); commercial EDA tool experience (100% free/pip-installable toolchain); clock-domain-crossing verification (both DUTs are single-clock); exhaustive formal coverage of the whole design (3 narrow properties on 1 of 2 DUTs); or that a mutation score of 100% means the RTL has no bugs (it means the specific, hand-selected injected defects were all caught).

2. Architecture -- How Everything Fits Together

Repository layout

test0/	minimal cocotb sanity test (AND gate) -- toolchain smoke test
phase1_uart/	UART TX DUT + verification environment (uart_env/)
phase2_llm/	same UART DUT, verified by an LLM-generated testbench instead
phase5_apb/	APB4-Lite register block DUT + verification environment (apb_env/) + formal properties (formal/)
mutation/	generic, DUT-agnostic mutation-testing engine
regression/	unified regression orchestrator
run_regression.py	one-command entry point
.github/workflows/	CI (GitHub Actions)

UART environment (phase1_uart/uart_env/)

File	Role
driver.py	Stimulus only -- send_byte(), reset(). Non-blocking: issues the start pulse and returns immediately.
monitor.py	Passively decodes frames from tx_serial/tx_busy, independent of what the driver intended to send.
reference_model.py	Pure Python, zero DUT/cocotb dependency -- computes the expected frame bits.
scoreboard.py	The only place a data-value pass/fail judgement happens, against the reference model.
checker.py	Protocol/timing invariants (start/stop bit, segment timing, frame duration) -- independent of the scoreboard, never told what byte was sent.
coverage.py	Functional coverage via cocotb-coverage.
sequences.py	Constrained-random stimulus generation.
harness.py	Shared setup/teardown wiring everything together.

APB environment (phase5_apb/apb_env/)

File	Role
driver.py	Stimulus only -- blocking write()/read(), matching APB's SETUP->ACCESS shape.
monitor.py	Reconstructs each completed transaction (addr, direction, data, PSLVERR) from bus observation alone.
reference_model.py	Pure Python state machine tracking ENABLE/DATA/BUSY/DONE.
scoreboard.py	Data-value AND register-map-legality (PSLVERR) comparison against the model.
checker.py	Pure APB bus-interface invariants (SETUP->ACCESS sequencing, PENABLE/PSEL relationship, signal stability, PREADY during ACCESS) -- deliberately does not know the address map.
latency_checker.py	Cycle-accurate check that an operation takes exactly OP_LATENCY cycles, via the busy_o observability port -- independent of the scoreboard's software-polling model.
coverage.py	Functional coverage.
sequences.py	Constrained-random value-generation primitives.

Register-map legality (which PADDR/direction combos are legal) lives in the **scoreboard**, not the protocol checker -- it's a fact about this specific device's address map, not a rule every APB4 slave must obey.

Mutation engine (mutation/)

mutation/framework.py + mutation/report.py: given a DUT directory and a manifest of Mutant (name, RTL file, defect category) and Mechanism (name, cocotb test module) objects, it runs `make clean && make VERILOG_SOURCES=<mutant> MODULE=<mechanism>` for every mutant x mechanism pair, classifies CAUGHT/SURVIVED from cocotb's own `FAIL=<n>` summary line, and produces a report. **Zero UART- or APB-specific code** -- both DUTs' `mutation_suite.py` are just manifests built on this same, unmodified engine.

Unified regression framework (regression/, run_regression.py)

File	Role
runner.py	Drives make for each test suite, extracts coverage YAML, runs lint, runs mutation_suite.py -- all via subprocess, treating each DUT directory as a black box.
formal.py	Drives SymbiYosys for each formal property, fetches sby automatically on first use, extracts counterexample artifacts on failure.
parsers.py	Reads cocotb's results.xml (JUnit XML), exported coverage*.yaml, and verible-verilog-lint's structured output -- machine-readable, not console scraping.
report.py	Builds one unified result object; renders regression_results/report.json and report.md.

run_regression.py sequences, in order: test0 sanity, UART directed/coverage-directed/random/checker-only, APB directed/random/checker-only/latency-only (9 suites total), lint on both RTL files, both DUTs' full mutation suites, and the 3 formal properties.

Formal verification layer (phase5_apb/formal/)

Three properties on apb_regblock only. Toolchain: yowasp-yosys + yowasp-yosys-smtbmc (Yosys compiled to WebAssembly, pip-installable) + z3-solver (pip-installable Z3), driven by real SymbiYosys (sby). All free, no sudo/apt, no billing. Properties are plain procedural assert/assume statements inside `always @(posedge PCLK)` blocks -- not SVA. Entirely independent of the simulation-side scoreboard/checkers: no Python, no cocotb, nothing shared except that both read the same `apb_regblock.v`.

Lint

Verible (verible-verilog-lint) checks style/structural issues on both RTL files. Findings are reported, not auto-fixed where fixing them would conflict with the mutation suite's "exactly one injected difference from golden" property (see Section 4).

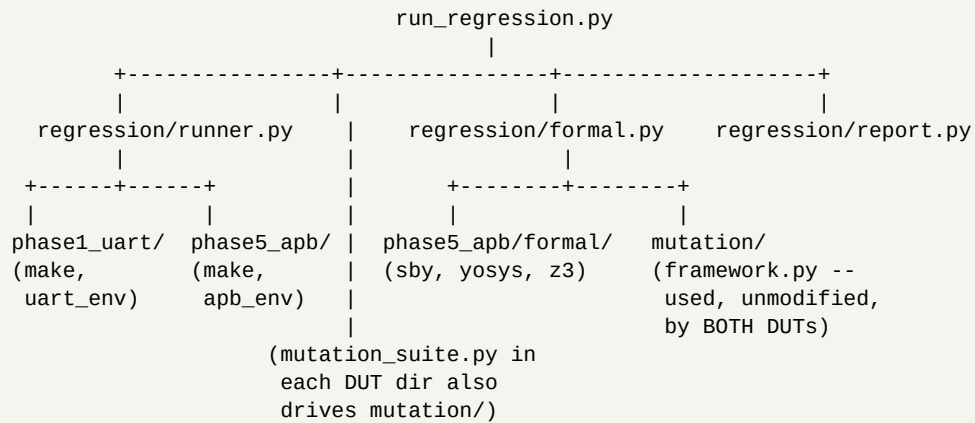
CI

.github/workflows/regression.yml runs run_regression.py on every push/PR to main on GitHub's free ubuntu-latest runners: installs Icarus Verilog, installs Python deps from requirements-lock.txt, fetches SymbiYosys, runs the full regression, uploads the report and any failure artifacts.

phase2_llm/ (separate research track)

The same UART TX RTL and mutants, verified instead by a cocotb testbench generated by an LLM (Gemini) from a natural-language prompt, scored with the same mutation-testing method for comparison against the hand-written suite. Deliberately kept out of the regression/CI/mutation-engine work above and untouched by every phase after its introduction. See Section 17.

Simple architecture diagram



Per-DUT chain (same shape for UART and APB):

```

driver --> DUT (RTL) --> monitor --+--> scoreboard --(vs)--> reference model
(stimulus) (uart_tx.v /          | (data value + APB legality)
            apb_regblock.v)    +--> protocol checker (structural/timing,
                                | no knowledge of data)
                                +--> [APB only] latency checker
                                +--> coverage (did we hit the right bins)
  
```

3. The Core Verification Story

This is the mental model to lead an interview answer with. Say it in this order -- it is literally the causal chain data flows through in this project.

1. DUT bug exists

A hand-injected RTL defect (a mutant) or, in principle, a real bug -- e.g. the APB busy-guard mutant: the START condition drops its !busy_reg check, so a retrigger while busy restarts the operation.

2. Stimulus

The driver (directed, coverage-directed, or constrained-random) issues legal transactions -- it has no idea whether the DUT underneath is correct.

Proves: the DUT was exercised with a given input sequence. Does NOT prove: correctness of the response, or that untested sequences would also work.

3. Monitor

Passively, autonomously decodes what actually happened on the DUT's outputs -- independent of what the driver intended.

Proves: an accurate observation of DUT behavior exists to check against. Does NOT prove: anything by itself -- it makes no judgement.

4. Reference model

An independent, pure-Python re-implementation of the DUT's expected behavior, computed the same sequence the driver issued.

Proves: an independent "expected answer" exists. Does NOT prove: the model itself is bug-free (mitigated, not eliminated, by mutation testing).

5. Scoreboard / protocol checker

Scoreboard compares monitor-observed data against the reference model (data-value judgement). Protocol/latency checkers independently verify structural and timing invariants, with zero knowledge of the expected data value.

Proves: for every transaction actually observed, values matched / invariants held. Does NOT prove: anything about untested internal states or unobserved scenarios.

6. Coverage

Records which defined structural/behavioral categories were actually exercised.

Proves: stimulus reached a given category at least once. Does NOT prove: correctness there (that's the scoreboard's job), or anything about categories not defined as bins.

7. Mutation testing

Deliberately re-injects specific defects into golden RTL and re-runs the whole environment, checking whether it actually fails.

Proves: the testbench actually detects each specific, hand-selected defect, attributed to the mechanism(s) that caught it. Does NOT prove: detection of defect classes no mutant represents.

8. Formal verification

For 3 chosen properties on the APB DUT, proves the property holds for every reachable state (k-induction) or over the full input space at every checked depth (BMC).

Proves: the stated property holds exhaustively, not just for tested stimulus. Does NOT prove: anything about the DUT's behavior not stated as one of those 3 properties.

9. Regression automation

run_regression.py ties all of the above into one reproducible command with a machine- and human-readable report, wired into real CI.

Proves: the whole verification suite is repeatable and automated, not hand-run once. Does NOT prove: anything additional about the DUT itself -- it's an engineering-process guarantee, not a correctness mechanism.

KEY TAKEAWAY: The single most interview-valuable sentence in this project: "No mechanism here proves everything -- each one has a specific, bounded scope, and I can tell you exactly what each one does and does not establish." That is the actual skill this project demonstrates.

4. Exact Measured Results

Source of truth: VERIFICATION_STRATEGY.md ("Measured results" section). Kept deliberately separate -- never combined into one artificial project score.

Test pass rate: 24/24 across 9 suites

Suite	Result	Notes
test0_sanity	1/1	Toolchain smoke test (AND gate)
uart_directed	2/2	Basic framing + timing
uart_coverage_directed	4/4	Broadened directed sweep + coverage + reset-mid-frame + data-latch
uart_random	2/2	Constrained-random, seed recorded per run
uart_checker_only	1/1	Protocol checker in isolation
apb_directed	10/10	Register R/W, illegal access, START guard, INTCLR, reset
apb_random	2/2	Constrained-random, seed recorded per run
apb_checker_only	1/1	Protocol checker in isolation
apb_latency_only	1/1	Latency checker in isolation

Mutation score (kept separate per DUT and per mechanism)

DUT	Combined	Per-mechanism breakdown
UART	9/9 (100.0%)	coverage_directed 9/9 (100%) scoreboard 7/9 (77.8%) protocol_checker 6/9 (66.7%)
APB	9/9 (100.0%)	scoreboard 9/9 (100%) latency_checker 2/9 (22.2%) protocol_checker 1/9 (11.1%)

The combined score is 100% for both, but the **per-mechanism spread** is the informative number: no single mechanism catches everything on its own -- each is narrow by design.

Functional coverage

- uart_coverage_directed: its 4 owned points/cross (tx_data_class, tx_mode, their cross, reset_mid_transmission) all **100.0%**.
- uart_random: its 2 owned points (idle_gap_class, back_to_back_chain_length) both **100.0%**.
- apb_random: all 5 points/cross **100.0%** (single coverage module -- its top figure is meaningful as-is).

GOTCHA: The aggregate cocotb-coverage "top" figure is **not** the meaningful number for the two UART modules, because coverage_db is a per-process singleton and each module's "top" includes points a different module owns. Always cite the **per-owned-point** percentage, never the raw aggregate, for UART.

Formal: 3/3 properties PASS

Property	Result	Proof type
P1 -- PSLVERR correctness	PASS	Bounded (BMC, depth 5) -- complete for this specific claim, since the property is purely combinational (no dependence on history)

Property	Result	Proof type
P2 -- no restart while busy	PASS	Unbounded (k-induction) -- true for all time
P3 -- BUSY bounded by OP_LATENCY	PASS	Unbounded (k-induction) -- true for all time

Lint findings (not fixed -- documented trade-off)

DUT	File	Findings
UART	uart_tx.v	4 (2 untyped parameters, 1 missing case default, 1 missing trailing newline)
APB	apb_regblock.v	4 (4 untyped parameters -- 2 of the original 6 findings were fixed before any mutants existed)

None are functional bugs. None are fixed because fixing them now would require regenerating every mutation-suite mutant to preserve the "exactly one injected difference from golden" property those files depend on -- a deliberate, explained trade-off, not an oversight.

Seed reproducibility

Verified directly, not assumed: re-running `make MODULE=test_uart_tx_random COCOTB_RANDOM_SEED=<seed>` (or the APB equivalent) with the same seed reproduces byte-for-byte identical randomized stimulus; a different seed produces a genuinely different sequence from the first transaction onward, for both DUTs. Every random suite's seed is generated **before** that suite runs (so it's recorded even on a hang or early failure), passed through as `COCOTB_RANDOM_SEED=<seed>`, and printed/reported.

CI status

Locally verified end-to-end, including from a byte-fresh virtual environment using only `requirements-lock.txt`.

Also directly confirmed running for real on GitHub's ubuntu-latest Actions runners against this exact repository, completing green end to end (installs Icarus Verilog, installs from `requirements-lock.txt`, fetches SymbiYosys, runs the full 9-suite/2-mutation-suite/3-formal-property regression, uploads the report). This is a real, observed remote CI run, not just a validated workflow file -- check the repository's Actions tab for the run history.

5. 5-Minute Recruiter Demonstration -- Step by Step

Practice this once out loud before any real call. Total run time for the live commands is under 3 minutes; the rest is narration.

STEP 1

ACTION	Open the repository root in a terminal and the README in an editor tab.
WHAT TO SHOW	The README's top paragraph and "What this is not" callout.
WHAT TO SAY	"This is a cocotb-based verification portfolio -- two different DUTs, a shared mutation-testing engine, formal verification, and CI. I deliberately documented what it does NOT claim, because overclaiming is the most common mistake in verification portfolios."
WHY IT MATTERS	Sets tone immediately: this is a credible, scoped project, not marketing.

STEP 2

ACTION	Terminal, repo root.
WHAT TO SHOW	The command about to run.
WHAT TO SAY	"Let me run the whole regression -- one command, covers both DUTs, mutation testing, and formal verification."
WHY IT MATTERS	Shows it's real and automated, not cherry-picked screenshots.

STEP 3

ACTION	Run the full regression.
WHAT TO SHOW	Live console output scrolling (test suites, then mutation, then formal).
WHAT TO SAY	"This runs 9 test suites, both mutation suites, and 3 formal proofs -- about 24 tests plus two full mutation sweeps."
WHY IT MATTERS	Concrete, not hand-wavy: exact numbers, not "lots of tests."

STEP 4

ACTION	Point at the generated report.
WHAT TO SHOW	regression_results/report.md -- specifically the "Overall: PASS" line and the per-suite table.
WHAT TO SAY	"Every suite passed -- 24/24 tests, and neither the coverage, mutation, nor formal sections collapse into one number, because that would hide information a reviewer actually needs."
WHY IT MATTERS	Demonstrates you understand WHY separation of results matters, not just that you have them.

STEP 5

ACTION	Demonstrate a mutation catch (see Section 7 for the full walkthrough).
WHAT TO SHOW	The APB mutant3 diff, then a live python mutation_suite.py run in phase5_apb/ showing that mutant CAUGHT.
WHAT TO SAY	"Here's a one-line RTL bug I deliberately injected -- a missing busy guard that lets a restart happen mid-operation. My scoreboard catches it, but interestingly my latency and protocol checkers don't -- and that's expected, because they're not designed to."
WHY IT MATTERS	This is the single best 90 seconds of the whole demo -- shows real engineering judgement, not just a green checkmark.

STEP 6

ACTION	Show the waveform (see Section 8 for signal-by-signal detail).
WHAT TO SHOW	make WAVES=1 then GTKWave open on the APB or UART FST file.
WHAT TO SAY	"This is the actual SETUP/ACCESS handshake on the bus -- PSEL, PENABLE, PADDR, and you can see PREADY tied high since this is a zero-wait-state slave."
WHY IT MATTERS	Proves you can actually read a waveform, not just generate a pass/fail report.

STEP 7

ACTION	Show formal verification.
WHAT TO SHOW	phase5_apb/VERIFICATION_PLAN.md's formal table, plus (if time) a live python run_regression.py --skip-mutation --skip-lint run.
WHAT TO SAY	"Two of these three properties are proven true for all time using k-induction -- not just tested, proven. The third is bounded BMC, but I can explain exactly why that's still a complete proof for its specific claim."
WHY IT MATTERS	Formal verification is the strongest differentiator in this project for an interviewer who knows the field -- don't rush this step.

STEP 8

ACTION	Explain the LLM experiment (see Section 17 for full script).
WHAT TO SHOW	phase2_llm/ directory listing, or just talk through it.
WHAT TO SAY	"As a side experiment I had an LLM generate a testbench for the same UART DUT from a prompt. First attempt caught 5 of 6 mutants; tightening the prompt to require explicit cycle-by-cycle timing checks got it to 6 of 6. It's a fun side note, not something I'm claiming is comparable to the real 9/9 suite -- different mutant set, no coverage, no independent checker architecture."
WHY IT MATTERS	Shows intellectual honesty and current awareness of LLM-assisted workflows without overclaiming.

Exact commands used above

```
# STEP 3 -- full regression
python run_regression.py

# STEP 5 -- mutation demo (from phase5_apb/)
cd phase5_apb
python mutation_suite.py

# STEP 6 -- waveform (from phase5_apb/ or phase1_uart/)
make clean
make WAVES=1
gtkwave sim_build/apb_regblock.fst      # or sim_build/uart_tx.fst

# STEP 7 -- formal only (from repo root)
python run_regression.py --skip-mutation --skip-lint
```

6. Live Demo Backup Plan

If: Regression takes longer than expected

Don't wait live. Say: "the full run including mutation testing usually takes a couple of minutes -- let me show you a report from a previous run while it finishes in the background," then open `regression_results/report.md` (or the GitHub Actions run) directly.

If: A test fails (shouldn't happen on golden RTL/main, but environments vary)

Stay calm, this is a good opportunity: "let's actually look at what the failure diagnostics captured" -- open `regression_results/failures/<suite>/simulator_output.log` and, if it's a single-make suite, the auto-captured waveform. This IS the designed behavior, not a broken demo.

If: GTKWave does not open

Skip to the formal counterexample story instead (Section 7/9) -- it also produces a real waveform artifact on failure, and you can narrate the SETUP/ACCESS signals from the code in `apb_env/checker.py` without a GUI.

If: Mutation testing takes too long

It's the slowest part by design (9 mutants x 3 mechanisms x full recompile each, per DUT). Pre-run it before the call and have `regression_results/report.md`'s mutation section, or the per-DUT VERIFICATION_PLAN.md table, open and ready. Narrate the APB mutant3 story (Section 7) from the markdown table instead of live output.

If: Formal verification takes too long

Each property normally proves in about a second against this DUT's small reachable state space -- if it hangs, it's almost certainly an environment/toolchain-fetch issue, not the proof itself. Fall back to the documented result table in Section 4 and explain BMC vs k-induction conceptually (Section 9).

If: The interviewer asks to see the code instead of a demo

Good -- open `apb_env/scoreboard.py` or `uart_env/checker.py` directly and walk the separation of concerns line by line. Section 27 has a 3-level depth script ready for exactly this.

If: The interviewer asks for a specific bug

Use the APB mutant3 story (Section 7) as your default -- it's the richest (real gap, investigated not force-fixed, later generalized into a formal property). If they want a simpler one, use UART mutant8 (data-latching bug, Section 10).

If: The interviewer asks to skip the full demo

Go straight to the 60-90 second pitch (Section 18), then let them steer with questions -- Section 21 has the hardest ones ready with concise answers.

7. Best Mutation Demo

Strongest available demonstration: **APB mutant3_start_missing_busy_guard**. It is the richest story in the project -- it initially survived **every single mechanism**, required real investigation (not a force-fix), and its defect class was later generalized into formal property P2. This is exactly the kind of story that separates a real engineer from someone who ran a script once.

Original RTL (apb_regblock.v, CTRL write handling)

```
ADDR_CTRL: begin
    enable_reg <= PWDATA[0];
    if (PWDATA[1] && enable_reg && !busy_reg) begin    // <-- guard: only start if NOT busy
        busy_reg <= 1'b1;
        op_count <= 8'h00;
    end
end
```

Mutated RTL (apb_regblock_mutant3.v)

```
ADDR_CTRL: begin
    enable_reg <= PWDATA[0];
    if (PWDATA[1] && enable_reg) begin                // <-- !busy_reg guard REMOVED
        busy_reg <= 1'b1;
        op_count <= 8'h00;
    end
end
```

What defect was injected

A single removed term: !busy_reg. With it gone, a CTRL write with START=1 while an operation is already in flight restarts the operation -- resetting op_count to 0 and effectively double-incrementing DATA by the time everything settles, instead of the correct single increment.

Which mechanism catches it

Mechanism	Verdict	Why
scoreboard	CAUGHT	DATA reads 0x02 (incremented twice) instead of the reference model's expected 0x01 -- a pure data-value defect, exactly the scoreboard's job.
latency_checker	SURVIVED	The operation still eventually completes in a latency-checker-visible window -- this mechanism only checks cycle count, not whether a restart occurred.
protocol_checker	SURVIVED	The APB bus-interface sequencing (SETUP->ACCESS, PSEL/PENABLE, PREADY) is completely unaffected -- this is a control-logic bug, not a protocol-interface bug, and the checker deliberately knows nothing about register semantics.

Overall verdict: CAUGHT (the mutation framework only needs one mechanism in the suite to catch a mutant for it to count as caught overall).

What the output looks like

```
$ python mutation_suite.py
...
mutant3_start_missing_busy_guard | control logic / missing guard
  latency_checker      : SURVIVED
  protocol_checker     : SURVIVED
  scoreboard          : CAUGHT
  Overall              : CAUGHT
```

The real story: it initially survived EVERYTHING

The first version of the test that was supposed to catch this (`test_apb_start_while_busy_is_ignored`) used a retrigger write of `data=0x02` (START only). But every CTRL write unconditionally updates `enable_reg <= PWDATA[0]` -- so the first START-only write already cleared `enable_reg` as a side effect of starting the operation. By the time a second START-only write was issued, `enable_reg` was already 0 regardless of whether the busy guard existed, so the guard's absence was never actually exercised. This was confirmed by direct simulation with internal-signal instrumentation (`busy_reg`, `op_count`, `done_reg`, `data_reg` traced cycle-by-cycle), and fixed by re-asserting ENABLE via its own separate write immediately before re-asserting START, with no idle gap anywhere in the sequence.

WHY THIS MATTERS: This later became **formal property P2** ("no restart while busy"), proven unbounded by k-induction -- the exact same defect class, but now proven impossible for any reachable state and any sequence of interfering writes, not just the one specific hand-crafted retrigger timing the simulation test uses. This progression (simulation test gap found -> fixed -> generalized into an exhaustive formal property) is the single strongest narrative thread in the entire project.

Exactly how to explain why mutation testing matters (script)

"Code coverage tells you a line executed. It doesn't tell you whether a test would actually notice if that line were wrong. Mutation testing answers the second question directly: I deliberately break the RTL in one specific way and check whether my own testbench notices. When it didn't -- like this case -- I didn't just add an assertion to force a pass. I instrumented the simulator, found the real reason the existing test structurally couldn't exercise the bug, and fixed the actual gap."

Why 9/9 does NOT mean the design is exhaustively verified

- 9/9 means all **9 hand-selected, hand-injected** defects were caught -- it says nothing about defect classes no mutant represents (e.g. multi-bit upsets, X-propagation, clock-domain-crossing issues -- none of which any mutant here models).
- A mutation score is only as good as the mutant set's realism and diversity -- both VERIFICATION_PLAN.md files include an explicit "mutant review" step precisely because of this limitation.
- Two candidate mutants were deliberately **rejected** as equivalent (indistinguishable from golden RTL in every reachable scenario) rather than added just to inflate the count -- see Section 10.
- Mutation testing measures the **testbench's** sensitivity to specific defects, not the DUT's absolute correctness -- it is a check on the checker, one layer removed from the RTL itself.

8. Waveform Demo

Exact steps

```
cd phase5_apb          # or phase1_uart
make clean
make WAVES=1
gtkwave sim_build/apb_regblock.fst      # or sim_build/uart_tx.fst

# To convert to VCD for tools that don't read FST:
fst2vcd sim_build/apb_regblock.fst -o sim_build/apb_regblock.vcd
```

In GTKWave: drag signals from the SST panel into the signal list to view them. This dumps an FST covering the full DUT hierarchy -- no Makefile changes required, cocotb's Icarus flow supports it out of the box.

APB signal reference

Signal	Meaning
PSEL	Slave select -- asserted by the master through both SETUP and ACCESS phases of a transfer.
PENABLE	Distinguishes SETUP (0) from ACCESS (1) -- the defining APB handshake signal.
PADDR[7:0]	Register address -- 0x00 CTRL, 0x04 STATUS, 0x08 DATA, 0x0C INTCLR; anything else is illegal.
PWRITE	1 = write, 0 = read.
PWDATA[7:0]	Write data, valid and stable through SETUP and ACCESS.
PRDATA[7:0]	Read data, driven by the slave during ACCESS.
PREADY	Tied high in this DUT -- a valid zero-wait-state APB4 slave; every transfer completes in exactly one ACCESS cycle.
PSLVERR	Asserted during ACCESS when the access is register-map-illegal (e.g. write to STATUS, read from INTCLR, or an unmapped address).

SETUP vs. ACCESS phase

Every APB transfer is exactly two cycles here (zero-wait-state): **SETUP** -- PSEL=1, PENABLE=0, address/write-data/direction already driven; then **ACCESS** -- PSEL=1, PENABLE=1 on the next clock edge, and the transfer completes (data captured on a write, PRDATA valid on a read) because PREADY is tied high.

What to point out in the waveform (script)

- "Watch PSEL go high one cycle before PENABLE -- that's SETUP, then PENABLE joins it for ACCESS. That two-phase shape is the entire APB protocol in one picture."
- "PREADY stays high the whole time -- this is a zero-wait-state slave, so every transfer is exactly one SETUP + one ACCESS cycle, no stalling."
- "Here's a write to an illegal address -- watch PSLVERR assert during ACCESS. My protocol checker doesn't know this is illegal, only my scoreboard does, because legality is a register-map fact, not a bus-protocol rule."
- "And here's busy_o -- that's my verification-only observability port. It's how my latency checker independently confirms the operation takes exactly OP_LATENCY cycles, without relying on software polling."

9. Formal Verification -- Interview Preparation

Simple explanation first

Simulation tests specific scenarios you thought to write. Formal verification uses a solver to mathematically check a property against **every possible** input sequence and state the DUT could ever reach -- not a sample, the whole space. If it says PASS, there is no possible counterexample. If it finds one, it hands you the exact failing trace.

Technically precise explanation

- **BMC (Bounded Model Checking)**: unrolls the design for a fixed number of cycles (a "depth") and asks an SMT solver whether the property can be violated within that many steps. If no violation exists up to that depth, it's a proof **up to that bound** -- not a proof for all time, unless the property provably has no dependence on history (see P1 below).
- **k-induction**: a two-step proof. Base case: check the property holds for the first k steps from reset. Induction step: assume the property has held for k consecutive arbitrary steps (not necessarily reachable from reset) and prove it must still hold on step k+1. If both steps succeed, the property is proven true **for all time, for any reachable state** -- a genuinely unbounded proof, not just "we didn't find a counterexample yet."

P1, P2, P3 -- exact status

Property	Statement	Proof	Result
P1	PSLVERR == (illegal_write illegal_read) during ACCESS, for every PADDR/PWRITE/PSEL/PENABLE combination	BMC, depth 5	PASS -- complete for this claim
P2	An accepted operation is never restarted while busy (any sequence of interfering CTRL writes)	k-induction	PASS -- unbounded
P3	BUSY is never held for more than OP_LATENCY consecutive cycles, for any reachable state	k-induction	PASS -- unbounded

Why P1 is bounded BUT complete for its specific claim

P1 is purely combinational -- PSLVERR depends only on the **current-cycle** inputs, never on any register. Proving it holds at every checked step (depth 5, though even depth 1 would suffice) is therefore a complete proof over the **entire interface input space** (all 256 PADDR values x both directions x every PSEL/PENABLE combination), even though the proof engine used is BMC, not induction. This is a property of **this specific claim** (no history dependence), not a general property of BMC results -- do not generalize "BMC proves things for all time."

Why P2/P3 are unbounded

Both are genuinely temporal (depend on history: "was busy last cycle," "how many cycles has busy been held"), so a fixed-depth BMC check could never rule out a counterexample appearing one cycle past the checked depth. k-induction closes that gap: once the induction step succeeds, the property is proven for arbitrarily long traces, not just the ones actually checked.

Assumptions used by the proofs (all explicit, none hidden)

- All three: the trace is assumed to begin with a genuine reset (`initial assume (!PRESETn)`). Without this, BMC is free to start from an arbitrary, physically-meaningless register state -- found by a real counterexample trace that simply never reset, not assumed in advance.

- P2 only: DATA-address writes are assumed not to occur while an operation is busy. A narrow, deliberate scoping decision -- a DATA write while busy is legal at the bus level and legitimately changes what value the eventual auto-increment operates on (already modeled in the reference model). Excluding it keeps P2 focused on the CTRL/START retrigger guard specifically. CTRL writes -- every ENABLE/START combination, including illegal ones -- remain completely unconstrained.

What formal proves / does not prove

Proves	Does NOT prove
The stated property holds for every reachable input at every cycle (k-induction: for all time; BMC: up to the checked depth, or exhaustively if history-independent)	Anything not stated as one of the 3 properties -- formal here covers 3 narrow, deliberately chosen claims, not the DUT's full functional correctness

Toolchain limitations, verified not assumed

- No SVA -- every property is a plain procedural assert/assume. property/sequence blocks were never tested against this toolchain's Yosys frontend.
- Hierarchical cross-module references (`dut . busy_reg`) are not reliably modeled -- confirmed with a trivial tautology (`busy_o == dut . busy_reg`, true by definition) that produced a **false counterexample**. Fixed by adding `data_o`, a real output port, and observing the DUT only through real ports from then on.

10. Mutation Testing -- Interview Preparation

What it is

Deliberately re-inject a specific, realistic RTL defect (a "mutant") into otherwise-golden RTL, then re-run the full verification environment against it. If any mechanism fails, the mutant is CAUGHT; if everything still passes, it SURVIVED -- meaning the environment has a real, concrete blind spot.

Why it is useful

Code/line coverage only tells you a line of RTL executed -- not whether any check would notice if that line were wrong. Mutation testing directly measures the testbench's actual defect-detection power, not just its reach.

What 9/9 means (and the limits of that claim)

All 9 hand-selected mutants per DUT were caught by at least one mechanism in the suite. It does not mean the RTL has no bugs, and it does not mean every possible defect class is covered -- only the ones a mutant was written to model. See Section 7 for the full nuance.

Per-mechanism attribution -- why it's the informative number

A combined 100% score hides which mechanism actually did the work. Reporting each mechanism's own catch rate separately (e.g. APB's protocol_checker at only 1/9) proves each mechanism is doing its own narrow job correctly, not redundantly duplicating another mechanism's coverage.

Equivalent mutants -- what they are and why they were excluded

An equivalent mutant is an RTL change that is provably indistinguishable from golden RTL in every reachable scenario -- no test could ever catch it, so including it would silently lower every mechanism's achievable score without adding any real signal.

DUT	Rejected mutant	Why it's equivalent
UART	Removing <code>bit_idx<=0/busy_reg<=0</code> from the reset branch	Both are unconditionally re-assigned by the IDLE state's own case branch on the very next reachable cycle -- traced through the RTL, confirmed no reachable scenario makes the difference externally observable.
APB	PREADY tied to PSEL alone instead of the constant 1	PSEL is already 1 throughout every ACCESS phase by construction (the master holds it asserted through SETUP and ACCESS), so the two expressions are indistinguishable in every cycle that actually matters.

Neither was added to its mutation suite -- the framework has an explicit `equivalent=True` field on `Mutant` to record a reviewed-and-rejected idea like these without running or scoring it.

Examples of defects caught by different mechanisms

Mutant	DUT	Caught by	Not caught by
mutant5_msb_first_bug (bit-order swap)	UART	scoreboard, coverage_directed	protocol_checker (correct timing, wrong data -- the checker categorically cannot see this)

Mutant	DUT	Caught by	Not caught by
mutant7_incomplete_reset	UART	coverage_directed (only, once added as a 3rd mechanism)	scoreboard, protocol_checker (test_uart_tx alone never resets mid-frame -- structurally can't exercise it)
mutant9_pready_never_asserted_on_write	APB	protocol_checker, scoreboard	latency_checker (writes just never complete -- not a timing-of-a-completed-op question)
mutant3_start_missing_busy_guard	APB	scoreboard only	latency_checker, protocol_checker (see Section 7 for the full story)

11. Coverage -- Interview Preparation

Functional coverage

Tracks which defined structural/behavioral categories of stimulus were actually exercised -- built with `cocotb-coverage` in this project. It answers "did we hit the right space," never "was the DUT's response correct" (that's the scoreboard's job).

Coverpoints and bins

A CoverPoint samples a value on every relevant transaction and buckets it into one of several defined bins. Example (UART `tx_data_class`): zero / all-one / alternating / walking-one / other -- 5 bins covering the structurally distinct byte-pattern classes, not all 256 raw byte values.

Constrained-random coverage

The random sequence generators (`sequences.py` in each DUT) **guarantee** every required scenario/coverage cell is hit at least once deterministically, while randomizing the specific values, ordering, and idle-gap timings within that guarantee. This is a stronger claim than pure random sampling -- coverage closure never depends on luck.

What 100% means in THIS project

For each module, its **own owned** CoverPoints/crosses reached 100% -- every defined bin was hit at least once by at least one transaction. It does not mean every possible input value was tried (e.g. `tx_data_class`'s "other" bin covers 246 possible byte values as a single bin).

What it does NOT mean

- It does not mean the DUT's response was correct in each bin -- a covered bin only says stimulus reached that category; correctness is the scoreboard's separate job.
- It does not mean exhaustive coverage of the input space -- it's a guaranteed-representative sample, a better-distributed sample than fixed directed vectors, not every possible value.
- The raw `cocotb-coverage` aggregate ("top") for the two UART modules is **not** the meaningful number, because `coverage_db` is a process-wide singleton and each module's aggregate includes points a different module actually owns/closes.

Why the coverage numbers are not an exhaustive state-space claim

Coverage bins are deliberately coarse, human-defined categories, not a state-space enumeration. That exhaustive claim is exactly what formal verification exists to make instead (Section 9) -- coverage and formal are complementary, not substitutes for each other.

12. Constrained-Random Testing

How stimulus is generated

uart_env/sequences.py and apb_env/sequences.py generate stimulus that **guarantees** every required coverage cell/scenario is exercised at least once (deterministically), while randomizing specific data values, ordering, and idle-gap/timing details within that guarantee -- not pure unconstrained randomness.

How seeds work

Each random suite generates a seed **before** that suite runs (so it's recorded even on a hang or early failure), passes it through as COCOTB_RANDOM_SEED=<seed>, and prints/reports it. cocotb's own RNG is seeded from this value, so the entire sequence of random decisions becomes deterministic given the seed.

Reproducibility

Verified directly (not assumed): running the same seed twice produces byte-for-byte identical randomized stimulus -- not just the same logged seed number, the actual transaction sequence. A different seed produces a genuinely different sequence from the first transaction onward, confirmed for both DUTs.

Corner-case injection

Corner cases aren't left to chance either -- the guaranteed-coverage-cell property means specific structurally interesting scenarios (e.g. 3+-frame back-to-back chains, all four illegal-access classes, every start_control outcome) are always exercised, with random data/ordering layered on top.

Why random testing is useful

Directed tests only exercise exactly what you thought to write. Random stimulus, especially guaranteed-coverage random stimulus, exercises combinations and orderings you didn't explicitly anticipate -- while still being fully reproducible for debugging.

How to reproduce a failure using a seed

```
# UART
cd phase1_uart
make clean && make MODULE=test_uart_tx_random COCOTB_RANDOM_SEED=284863992

# APB
cd phase5_apb
make clean && make MODULE=test_apb_regblock_random COCOTB_RANDOM_SEED=1112575254
```

(These example seeds are the ones recorded in VERIFICATION_STRATEGY.md from a documented run -- any seed printed by a real run works the same way. Always make clean first; GNU Make's mtime-based rebuild check has, once, been observed to reuse a stale compiled binary otherwise -- see LESSONS_LEARNED.md.)

13. Scoreboard / Reference Model

Why the reference model is independent

Each reference model (`uart_env/reference_model.py`, `apb_env/reference_model.py`) is pure Python with zero DUT/cocotb dependency -- it re-implements the DUT's expected behavior from the specification, not by mirroring the RTL's implementation. If the model were derived from the RTL itself, a shared misunderstanding could make both wrong in the same way and the scoreboard would never notice.

What the scoreboard compares

UART: monitor-decoded byte and per-segment timing samples against the reference model's expected frame bits. APB: monitor-observed PRDATA/PSLVERR/PWDATA against the model, including register-map legality (is this address/direction combination even legal).

Why the scoreboard does not own latency checking (APB)

The reference model is deliberately "software-realistic" -- it only learns an operation finished when the test polls and observes `BUSY=0`, exactly the way real firmware would. A transaction-level model that only knows "eventually finished" cannot, by construction, distinguish a correct latency from an off-by-one defect landing between two polls. That's precisely why `ApbLatencyChecker` (cycle-accurate, fully independent) and formal property P3 (exhaustive) exist as separate mechanisms instead of folding this into the scoreboard.

Why separation of concerns matters

Every mechanism in this project checks exactly one thing and is blind to everything else on purpose: the scoreboard never knows about timing, the protocol checker never knows the expected data value, the latency checker never knows about data correctness. This is what makes the per-mechanism mutation-attribution table (Section 4) meaningful -- if any mechanism quietly did another's job, the attribution would be misleading, and a weakness in one mechanism could be masked by accidental overlap with another.

14. Protocol Checker vs. Scoreboard vs. Latency Checker

Mechanism	What it checks	What it does NOT check	Example defect it catches
Scoreboard	Data-value correctness against an independent reference model (UART: decoded byte; APB: PRDATA/PSLVERR/PWDATA, plus register-map legality)	Timing/cycle counts; bus-interface sequencing rules	APB mutant2: CTRL readback bit-swap (ENABLE/START swapped) -- a pure data-value bug
Protocol checker	Data-independent structural/timing invariants that apply to any legal transfer (UART: start/stop bit value, segment constancy; APB: SETUP->ACCESS sequencing, PENABLE/PSEL relationship, signal stability, PREADY during ACCESS)	The actual data value transmitted or the register-map legality of an address	APB mutant9: PREADY never asserted on write -- writes structurally never complete, a pure interface bug
Latency checker (APB only)	Cycle-accurate: does every observed operation take exactly OP_LATENCY cycles, measured via the busy_o observability port	Whether the resulting DATA value is correct, or whether the operation was legitimately triggered	APB mutant4/mutant5: operation completes 1 cycle early/late

TALKING POINT: Interview gold: mutant3 (Section 7) is CAUGHT only by the scoreboard, and SURVIVED by both the latency checker and the protocol checker -- not because those two are weak, but because a busy-guard bypass is a pure control-logic/data-value defect, entirely outside what either of them is designed to see. Being able to explain *why* a mechanism correctly misses something is more impressive than a mechanism that (accidentally) catches everything.

15. UART -- What I Should Know

Q: How does the transmitter work?

A: A small FSM: IDLE -> START -> 8 DATA bit states (LSB first) -> STOP -> back to IDLE. Each state holds `tx_serial` at a fixed value for exactly `CLKS_PER_BIT` clock cycles before advancing.

Q: What is the start bit?

A: A single bit held at logic 0 for `CLKS_PER_BIT` cycles, signaling the start of a frame to a receiver (idle line is logic 1).

Q: What are the 8 data bits?

A: The byte to transmit, sent LSB (bit 0) first, each bit held constant for `CLKS_PER_BIT` cycles -- this is standard UART convention, and getting the bit order backwards was exactly one of the injected mutants (mutant5).

Q: What is the stop bit?

A: A single bit held at logic 1 for `CLKS_PER_BIT` cycles, marking the end of the frame and returning the line to its idle level.

Q: What is `CLKS_PER_BIT`?

A: A configurable parameter setting how many DUT clock cycles each UART bit period lasts -- effectively sets the baud rate relative to the DUT clock.

Q: How is bit timing checked?

A: The protocol checker (`uart_env/checker.py`) samples every clock cycle of every segment -- no fixed-time-offset assumption -- and fails if any segment is not constant for exactly `CLKS_PER_BIT` cycles. This is a genuinely cycle-by-cycle check, not a mid-bit sample.

Q: How is data correctness checked?

A: The scoreboard compares the monitor-decoded byte against the reference model's expected value for that transaction -- entirely separate from the timing check above.

Q: What happens during busy?

A: `tx_busy` is asserted for the full frame duration and deasserts on the cycle the stop bit's last cycle commits. A stuck-high `tx_busy` is exactly mutant4, caught by all three mechanisms.

Q: What happens if data changes mid-frame?

A: Correct RTL captures `tx_data` once into an internal register at frame start; the transmitted byte must be unaffected by any later change to `tx_data`. This was a genuine, structurally-undetected gap (mutant8) until a dedicated test was added -- see Section 10.

Q: What reset behavior is tested?

A: A reset asserted mid-frame must return the DUT safely to idle (`tx_busy=0`, `tx_serial=1`) and transmit correctly afterward. This is tested directly, and a related bug (an unbounded wait loop that hung instead of failing when reset handling was broken) was found and fixed during this exact test's development -- see `LESSONS_LEARNED.md`.

16. APB -- What I Should Know

Signals

PCLK, PRESETn (active-low), PADDR[7:0], PWRITE, PSEL, PENABLE, PWDATA[7:0] (inputs); PRDATA[7:0], PREADY, PSLVERR (outputs, PREADY tied high); plus busy_o and data_o, two verification-only observability ports (not part of the APB interface).

SETUP / ACCESS

SETUP: PSEL=1, PENABLE=0, address/data/direction driven. ACCESS: PSEL=1, PENABLE=1 on the next cycle, transfer completes (PREADY tied high = zero-wait-state).

Registers

Addr	Register	Access	Semantics
0x00	CTRL	R/W	bit0 ENABLE (persistent); bit1 START (write-1 pulse, never stored, always reads 0); bits[7:2] reserved, always read 0
0x04	STATUS	RO	bit0 BUSY, bit1 DONE; a write raises PSLVERR
0x08	DATA	R/W	8-bit operand/result
0x0C	INTCLR	WO	bit1=1 clears DONE (write-1-to-clear); a read raises PSLVERR
other	--	--	raises PSLVERR on read or write

Invalid addresses / PSLVERR

PSLVERR = illegal_write || illegal_read, purely combinational on the current-cycle inputs -- exhaustively proven correct for all 256 addresses x both directions by formal property P1 (Section 9).

START / BUSY / DONE / operation latency

Writing CTRL with START=1 starts the operation **only if ENABLE was already committed from a prior write** -- the condition checks the registered enable_reg, not the bit just written in the same access, so writing ENABLE and START together in one write does NOT start anything. Once accepted, BUSY sets; after exactly OP_LATENCY (4) cycles, DATA increments by 1, BUSY clears, DONE sets. A START while disabled or already busy is ignored (proven exhaustively by formal property P2).

Likely interview questions and concise answers

Q: Why does register-map legality live in the scoreboard, not the protocol checker?

A: Because it's a fact about THIS device's specific address map, not a rule any APB4 slave must obey -- the protocol checker only covers interface rules that apply to any APB4 slave.

Q: Why is PREADY tied high?

A: This DUT is a valid zero-wait-state APB4 slave -- every transfer completes in exactly one ACCESS cycle, which simplifies the protocol checker's job (no wait-state sequencing to verify) while remaining a fully legal APB4 implementation.

Q: Why doesn't writing ENABLE and START together start an operation?

A: The START condition checks the registered enable_reg, which reflects the PREVIOUS write, not the bit just arriving in the same write -- so ENABLE must already be committed before a START in a later write can take effect. This is deliberate, realistic register-semantics behavior, and it's directly tested.

Q: What are busy_o and data_o and why do they exist?

A: Verification-only observability ports, not part of the APB interface. busy_o lets the latency checker verify OP_LATENCY cycle-accurately without relying on software polling reads. data_o was added specifically because this formal toolchain doesn't reliably model hierarchical references into internal registers -- exposing a real port worked around a confirmed tool limitation.

17. The LLM Experiment

VERIFIED: These numbers (5/6 and 6/6) were re-measured directly for this handbook by re-running both LLM-generated testbenches against all 6 original mutants in phase2_llm/ -- not taken from a stale record. mutant3_short_start_bit is the exact one that survived v1 and was caught by v2.

What phase2_llm does

A side experiment, deliberately separate from the main regression/CI/mutation-engine work: the same UART TX golden RTL and the same 6 original Phase-3 mutants, but verified by a cocotb testbench **generated by an LLM** (Gemini) from a natural-language prompt instead of hand-written. Scored with the same CAUGHT/SURVIVED mutation methodology, for comparison.

How the LLM-generated testbench was produced

phase2_llm/gen_llm_test.py reads the golden RTL, embeds it in a prompt describing the DUT and asking for a thorough cocotb testbench checking framing, data order, bit timing, and busy-flag behavior, and saves the model's response directly as a test file. No hand-editing of the generated code.

v1 result: 5/6

test_uart_tx_llm.py caught 5 of 6 mutants. **mutant3_short_start_bit survived** (start bit held one cycle too short).

What bug it missed, and why

v1's frame monitor samples each bit at a fixed `Timer()`-based offset from the point `tx_serial` first goes low -- it checks the start bit's value once, then advances by a fixed `clks_per_bit` delay per bit and samples again. It **never explicitly asserts that a segment holds constant for the full `CLKS_PER_BIT` duration** -- only that a single sampled point mid-bit has the right value. A start bit that's one cycle short doesn't change what value gets sampled at those fixed offsets closely enough to trip any assertion, so the defect passes through undetected.

v2 result: 6/6

test_uart_tx_llm_v2.py caught all 6 mutants, including mutant3.

What changed in the prompt / test

The regenerated prompt (in `gen_llm_test.py`) added one explicit instruction: *"Explicitly verify precise bit timing: for every bit period ..., sample the signal on every clock cycle within that period and assert it holds a constant value for exactly `CLKS_PER_BIT` cycles. Do not assume bit boundaries using fixed time offsets -- measure directly, cycle by cycle."* The resulting v2 code implements exactly that: a `verify_bit_period()` helper that loops for `CLKS_PER_BIT` cycles, asserting the value is unchanged on every single cycle. That structural change -- not a smarter model, a more precise prompt -- is what catches mutant3.

Why 6/6 is NOT comparable to the current 9/9

- Different, smaller mutant set: the LLM experiment only ever tests the original 6 Phase-3 mutants, not the full 9 (mutant7/8/9 -- reset handling, data-latching, long-start-bit -- were added later in Phase 4 and never run against the LLM testbench).
- No separation of concerns: the LLM testbench is a single monolithic test checking data value and timing together, not an independent scoreboard + protocol checker architecture -- so there's no per-mechanism attribution to compare.

- No functional coverage, no constrained-random stimulus, no regression/CI integration -- it's a single test file, not a verification environment.
- 6/6 measures "did one LLM-generated test file happen to catch 6 specific bugs," not "is this a comparable verification methodology" -- those are different claims.

Why it remains a side experiment

It answers a narrow, interesting question (can an LLM one-shot a reasonable testbench, and does prompt precision matter) without pretending to be a substitute for the structured, multi-mechanism environment the rest of the project builds.

What I should say in an interview

"As a side experiment, I had an LLM generate a cocotb testbench for the same UART DUT from a prompt. The first version caught 5 of 6 mutants -- it missed a one-cycle-short start bit because it sampled at fixed time offsets instead of verifying exact segment duration. I tightened the prompt to explicitly require cycle-by-cycle duration checking, and the regenerated version caught all 6. It's a nice illustration that prompt precision directly affects test quality, but I don't treat it as comparable to my hand-built 9-mutant, 3-mechanism suite -- different, smaller mutant set, no independent checker architecture, no coverage, no CI."

What I must NOT claim

DO NOT SAY: Never say "the LLM testbench is as good as my hand-written one" or cite 6/6 next to 9/9 as if they're the same scale. Never claim the LLM experiment used the full 9-mutant set. Never claim v2's fix was "a smarter model" -- it was a more precise prompt, a meaningfully different and more honest claim.

18. 60-90 Second Project Explanation

"I built a complete verification portfolio around two structurally different digital designs -- a UART transmitter and an APB4-Lite register block -- using cocotb, the Python-based verification framework. Each DUT has its own full environment: a driver for stimulus, a passive monitor, an independent reference model, a scoreboard that checks data correctness, and a separate protocol checker that checks structural and timing invariants without ever knowing the expected data value -- so each mechanism has one job and no overlap.

On top of that, I built a DUT-agnostic mutation-testing engine -- the same, unmodified code drives both DUTs -- which deliberately injects specific RTL defects and measures whether my own testbenches actually catch them. That process found two real gaps in my verification environment, which I investigated and fixed rather than just adding assertions to force a pass. Both DUTs now score 9 out of 9 on their mutation suites, and I can explain exactly which mechanism catches which defect class and why.

For the register block, I went a step further and formally verified two properties using k-induction -- proven true for all time, not just tested -- plus a third proven complete via bounded model checking for its specific combinational claim. Everything runs through a single-command regression framework wired into real GitHub Actions CI, using entirely free, open-source tooling -- no commercial EDA licenses anywhere.

The thing I'm most proud of isn't any single metric -- it's that every claim in the project's documentation is scoped honestly: I wrote out explicitly what each mechanism proves and does not prove, so nothing gets overclaimed."

19. 30-Second Version

"I built full verification environments -- driver, monitor, scoreboard, protocol checker, coverage -- for two different RTL designs using cocotb, then layered a custom mutation-testing engine on top to prove my own testbenches actually catch bugs, and added formal verification -- k-induction, proven for all time -- on one of them. Everything runs through one command, tied into real CI on GitHub Actions, using only free tooling. Both designs score 9 out of 9 on mutation testing, and I can walk you through exactly what that does and doesn't prove."

20. "Tell Me About Your Project" Answer

"So the project that I think best shows how I actually work is a verification portfolio I built around two small RTL designs -- nothing exotic, a UART transmitter and a little APB register block, because I wanted the focus to be on the verification methodology, not on building an impressive DUT.

What I actually spent most of my time on was making sure each piece of the environment only did one job. My scoreboard checks data values and nothing else. My protocol checker checks timing and bus sequencing and is never even told what byte was supposed to be sent. That separation sounds like a small thing, but it's what let me build a table later showing exactly which mechanism catches which kind of bug -- which turned out to be a really useful thing to be able to point at.

The part I'd actually want to talk through with you, if you're interested, is a bug I found using mutation testing on the APB design. I deliberately broke the busy-guard logic -- one line, removed a condition -- and re-ran everything. It survived every single check the first time. Instead of just accepting that, I dug into why, and it turned out my test itself had a stimulus gap, not my environment -- a side effect of an earlier write was masking the exact scenario I needed to trigger. I fixed the test, confirmed it now caught the bug, and later turned that same defect class into a formal property that's proven true for all time instead of just tested for one scenario.

That's honestly the story I'd want an interview to go into, because it's not really about the tools -- it's about not trusting a green checkmark until you understand why it's green."

21. Top Technical Interview Questions

10 hardest questions

Q: A mutation score of 100% -- doesn't that mean your design has no bugs?

A: No. It means all 9 hand-selected, hand-injected defects were caught by at least one mechanism. It says nothing about defect classes no mutant represents. I was careful to document this distinction explicitly rather than let 100% imply more than it does.

Q: Why didn't you use SystemVerilog Assertions instead of Python checkers?

A: I verified directly, not assumed, that Icarus Verilog 12.0 fails to parse concurrent/temporal SVA -- property, assert property, ##, |-> sequences all error out. Simple immediate assertions do compile, but the invariants I needed (a segment constant for exactly CLKS_PER_BIT cycles) are inherently temporal. Given this toolchain's verified limits, a cocotb concurrent checker was the strongest mechanism actually available, not a default choice.

Q: Why is P1 called a complete proof if it's only BMC, not k-induction?

A: Because PSLVERR is purely combinational -- it depends only on current-cycle inputs, never on any register. Proving it at every checked BMC step is therefore a complete proof over the entire interface input space, since there's no history dependence to leave unchecked past the depth. This is a property of that specific claim, not a general property of BMC.

Q: How do you know your reference model itself isn't buggy in the same way as the DUT?

A: I can't eliminate that risk entirely -- I document it explicitly as a known limitation of the scoreboard mechanism. What mitigates it: the model is pure Python written independently from the RTL implementation (from the spec, not by mirroring the Verilog), and mutation testing exercises the model too, since a wrong model would fail to distinguish golden RTL from an injected defect on that specific behavior.

Q: Why does register-map legality live in the scoreboard and not the protocol checker for APB?

A: Because it's a fact about this specific device's address map, not a rule any APB4 slave must obey. The protocol checker covers only interface rules any compliant slave would need to follow -- conflating the two would misrepresent an implementation detail as a protocol requirement.

Q: Your mutant3 story -- how did you know the test gap was real and not just bad luck?

A: I instrumented internal RTL signals (busy_reg, op_count, done_reg, data_reg) and traced them cycle-by-cycle through direct simulation, rather than guessing. That showed the first START-only write already cleared enable_reg as a side effect, so the second write's busy-guard-bypass scenario was structurally never reached -- confirmed, not assumed.

Q: Why do you exclude DATA writes while busy in formal property P2?

A: Because a DATA write while busy is legal at the bus level and legitimately changes what value the eventual auto-increment operates on -- already modeled in my reference model. An early version of P2 without this exclusion produced a counterexample that was really just this legal, already-understood behavior, not a restart bug. I scoped the property to what it's actually about instead of weakening it to force a pass.

Q: What's the actual difference between what mutation testing and formal verification each give you here?

A: Mutation testing proves my testbench detects specific, hand-picked concrete defects -- it's empirical and bounded to the mutant set. Formal proves a stated property holds for every reachable state and every possible interfering input, exhaustively -- but only for the 3 properties I chose to state. They're complementary: mutation testing found the busy-guard gap as a concrete failure; formal later proved the fixed behavior holds for all time, not just for that one scenario.

Q: Is your CI actually running, or just configured?

A: It's actually running -- I've confirmed a real GitHub Actions run on ubuntu-latest completing green end to end, not just a validated YAML file. It installs Icarus Verilog and all dependencies from a pinned lock file, fetches the formal toolchain, and runs the full 24-test/2-mutation-suite/3-formal-property regression.

Q: What would you add if you had another two weeks?

A: UART RX to get a full-duplex loopback DUT, integrating the formal properties as a 4th mutation-testing mechanism (currently deliberately deferred, not integrated), and a CLKS_PER_BIT elaboration-time sweep -- I verified this isn't a clean drop-in with the current Makefile setup and scoped it out rather than rush a fix.

Likely follow-up questions by topic

Topic	One-line answer
cocotb vs SystemVerilog/UVM	cocotb lets you write testbenches in Python against any HDL via a simulator's VPI/FLI interface -- no class-library boilerplate, direct access to the whole Python ecosystem (I used cocotb-coverage for functional coverage). UVM's phasing/factory/sequencer machinery adds real value at large-scale, multi-team SoC verification; at this DUT's scope, plain cocotb classes already deliver full separation of concerns without that overhead -- I made that call deliberately, not by default.
constrained random	My generators guarantee every required coverage cell is hit deterministically while randomizing values/ordering within that guarantee -- so coverage closure never depends on getting lucky with a seed.
coverage	Functional coverage only, via cocotb-coverage -- CoverPoints and crosses, exported as YAML after each run. I explicitly do not treat the aggregate number as meaningful for modules that don't own every CoverPoint (process-wide singleton behavior).
scoreboards	One per DUT, comparing monitor-observed values against an independent, pure-Python reference model -- for APB, this includes register-map-legality checking, not just data values.
protocol checking	Data-independent structural/timing invariants only -- for APB, pure APB4 bus-interface rules any slave must follow; never knows the address map's legality.
mutation testing	A DUT-agnostic engine (mutation/framework.py) drives make with a swapped-in mutant RTL file and a target test module, classifies CAUGHT/SURVIVED from cocotb's own summary line.
formal verification	SymbiYosys + Yosys (WASM build) + Z3, entirely free/pip-installable. 3 properties on the APB DUT, none SVA -- plain procedural assert/assume.
k-induction	A two-step proof (base case + induction step) that, once both succeed, proves a property true for all time for any reachable state -- not just up to a checked depth.
BMC	Unrolls the design for a fixed depth and checks for a violation within that many steps -- a bounded proof, unless the property provably has no history dependence (like P1).
APB	APB4-Lite register block, zero-wait-state (PREADY tied high), CTRL/STATUS/DATA/INTCLR registers, genuine multi-cycle operation with configurable OP_LATENCY.
UART	Configurable-CLKS_PER_BIT 8N1 transmitter, async open-loop framing, non-blocking driver.
CI	GitHub Actions, real ubuntu-latest runners, runs the identical run_regression.py used locally -- verified actually executing and passing.
reproducibility	Every random suite's seed is generated before the run, logged, and passed through COCOTB_RANDOM_SEED -- verified to reproduce byte-for-byte identical stimulus.
debugging	Every non-obvious timing/behavioral claim in this project was verified by instrumenting the simulator directly, never hand-derived and trusted -- see LESSONS_LEARNED.md for 7+ concrete examples with root cause.

22. The Biggest Weaknesses / How to Answer Them

No SystemVerilog / UVM

Likely question: "Why not use industry-standard SV/UVM instead of cocotb?"

Honest answer: I don't have production SV/UVM experience -- this project used cocotb (Python) deliberately, and I verified directly that this toolchain's Icarus Verilog build doesn't even parse SVA, so SV assertions weren't a realistic option here regardless.

Pivot: "What I can show is that I understand the underlying verification concepts UVM formalizes -- driver/monitor/scoreboard/coverage separation, reusable components -- independent of which language expresses them. cocotb is increasingly used in real industry flows too, especially for mixed Python/HDL environments."

Simple DUTs

Likely question: "These are pretty small designs -- how does this scale to a real SoC?"

Honest answer: Both DUTs are intentionally small so the focus stays on verification methodology, not RTL complexity. Neither is production-scale.

Pivot: "What I'd point to instead is that the architecture demonstrably generalizes -- the mutation engine and regression framework are the literal same, unmodified code driving two structurally different DUTs. That's the part that scales, not the DUT size."

No commercial EDA tools

Likely question: "Have you used VCS, Questa, or a commercial formal tool?"

Honest answer: No -- this entire project deliberately uses only free, pip-installable tooling (Icarus Verilog, SymbiYosys/Yosys/Z3) so it's reproducible by anyone with no license.

Pivot: "The underlying concepts -- constrained-random, functional coverage, k-induction, BMC -- are the same regardless of vendor. I'd expect the ramp-up to a commercial tool's specific UI/flags to be fast given I already understand what each step is actually trying to accomplish."

Limited clock-domain-crossing (CDC) coverage

Likely question: "How did you handle CDC verification?"

Honest answer: I didn't -- both DUTs are single-clock designs, so there's no CDC to verify here. This project doesn't claim any CDC methodology.

Pivot: "I'd want to be upfront that this is a real gap in my direct experience. What transfers is the general discipline -- verify empirically, never assume a timing relationship holds without instrumenting it, which is exactly the mindset CDC verification also demands."

LLM experiment limitations

Likely question: "Isn't the LLM testbench result kind of a stretch?"

Honest answer: It's a small, honestly-scoped side experiment -- 6 mutants, one test file, no coverage or independent checker architecture -- not comparable to the main 9-mutant/3-mechanism suite, and I say so explicitly.

Pivot: "I included it specifically because I could speak honestly about its limits, not because it's impressive on its own. What I'd actually highlight is what it demonstrates about prompt precision affecting test quality -- a genuinely current, relevant skill."

RULE: Never deny any of these gaps exist. The strongest version of every answer above names the limitation plainly in the first sentence, then pivots to what was actually demonstrated -- in that order.

23. Career / Resume Section

3 strong project title options

- cocotb-Based Digital Design Verification Portfolio (UART + APB4-Lite)
- Multi-DUT Functional and Formal Verification Framework with Mutation-Testing Validation
- Verification Methodology Portfolio: Constrained-Random, Coverage, Mutation Testing, and Formal Proofs

5 detailed resume bullets

- Built cocotb-based verification environments (driver/monitor/scoreboard/protocol checker/functional coverage) for two structurally different RTL designs (UART TX, APB4-Lite register block), achieving 100% functional coverage closure on all defined coverage points via guaranteed-representative constrained-random stimulus.
- Designed a DUT-agnostic mutation-testing engine (18 hand-reviewed mutants across both designs, zero shared code with either DUT) achieving 100% combined mutation score on both designs; investigation of two initially-surviving mutants found and fixed one real test-stimulus gap and one real environment gap.
- Formally verified two safety properties by k-induction (proven true for all time) and one by bounded model checking (proven complete for its combinational claim) on a register block, using a fully free, pip-installable toolchain (SymbiYosys + Yosys + Z3) -- no commercial EDA license.
- Built a unified, one-command regression framework (9 test suites, 2 mutation suites, 3 formal proofs) with automatic failure-diagnostic capture (waveforms, formal counterexamples) and seed-based reproducibility verified byte-for-byte, wired into real GitHub Actions CI confirmed running and passing.
- Root-caused 7+ real tool and framework bugs during development (a GNU Make PWD-inheritance bug, a formal-tool hierarchical-reference limitation, an over-strong formal property caught by its own counterexample) through direct simulator instrumentation, documented with full root-cause analysis rather than symptom-level fixes.

2-line resume version

Built cocotb verification environments for two RTL designs with driver/monitor/scoreboard/coverage/mutation-testing/formal-verification layers, achieving 100% mutation score and 2 unbounded formal proofs, automated via one-command regression and GitHub Actions CI.

ATS-friendly version (keyword-dense)

Digital design verification project using cocotb, Python, Verilog, Icarus Verilog, SymbiYosys, Yosys, Z3, GitHub Actions CI. Skills: functional verification, testbench development, driver/monitor/scoreboard architecture, protocol checking, functional coverage, constrained-random verification, mutation testing, formal verification, k-induction, bounded model checking (BMC), APB protocol, UART protocol, regression automation, waveform debugging (GTKWave), RTL lint (Verible), reproducible testing, git version control.

Version for a VLSI verification internship application

Independently designed and implemented functional verification environments for two digital designs using cocotb (UART transmitter and an APB4-Lite register block), including driver/monitor/scoreboard/protocol-checker architecture, functional coverage, and constrained-random stimulus. Validated testbench effectiveness with a self-built mutation-testing framework (100% mutation score, both designs) and performed formal property verification (k-induction, BMC) on register-block control logic. All work is automated through a unified regression suite integrated with GitHub Actions CI.

Version for a general ECE/VLSI fresher resume

Personal project: built and verified two digital hardware designs (a UART transmitter and an APB bus register block) using industry verification concepts -- testbenches, functional coverage, constrained-random testing, and formal (mathematical) proof techniques -- with a fully automated, one-command test suite and continuous integration on GitHub.

24. LinkedIn / GitHub Project Description

Short GitHub description

cocotb-based digital design verification portfolio: UART + APB4-Lite DUTs, driver/monitor/scoreboard, constrained-random stimulus, DUT-agnostic mutation-testing engine, formal verification (k-induction), unified regression + CI

Longer GitHub README-style description

A cocotb-based digital design verification project built around two genuinely different DUTs -- a UART transmitter and an APB4-Lite register block -- each with its own full verification environment (driver, monitor, reference model, scoreboard, protocol checker, functional coverage, constrained-random stimulus), a shared DUT-agnostic mutation-testing engine, a unified reproducible regression runner with CI, and a small formal-verification layer on the APB DUT. Includes a separate research track comparing a hand-written UART testbench against one generated by an LLM, scored by the same mutation suite. Every verification mechanism has a specific, bounded, explicitly documented scope -- this project does not claim to be "fully verified."

LinkedIn project description

I built a digital design verification portfolio to go deep on verification methodology rather than RTL design: two structurally different DUTs, each with a full cocotb environment (driver/monitor/scoreboard/protocol checker/coverage), a mutation-testing engine I built to validate my own testbenches (100% score, two real gaps found and fixed along the way), and formal verification proving two properties true for all time via k-induction. Everything runs through one command and real GitHub Actions CI, using entirely free, open-source tooling. Full writeup and source: github.com/sanjanaa2102/vlsi-verification-project

1-line portfolio description

Two-DUT cocotb verification portfolio with a self-built mutation-testing engine (100% score) and formal verification (k-induction) -- fully automated, free tooling, real CI.

25. Recruiter Demo Cheat Sheet

One page. Nothing else needed on screen during a live demo.

Commands

```
python run_regression.py                # full regression, repo root
cd phase5_apb && python mutation_suite.py # mutation demo
cd phase5_apb && make clean && make WAVES=1 && gtkwave sim_build/apb_regblock.fst
python run_regression.py --skip-mutation --skip-lint # formal only, repo root
```

Files to open

- README.md -- opening pitch
- VERIFICATION_STRATEGY.md -- the mechanism table + measured results
- phase5_apb/apb_regblock_mutant3.v vs apb_regblock_golden.v -- the mutation demo diff
- regression_results/report.md -- generated after a live run

Results to show

- 24/24 tests passed, 9 suites
- 9/9 mutation score, both DUTs, with per-mechanism breakdown
- 3/3 formal properties PASS (2 unbounded k-induction, 1 complete BMC)
- Real green GitHub Actions run

5 key things to say

- "Every mechanism here has a specific, documented, bounded scope -- I can tell you exactly what each one proves and doesn't."
- "The mutation engine and regression framework are literally the same, unmodified code driving both DUTs."
- "Two of three formal properties are proven true for all time, not just tested."
- "Mutation testing found two real gaps in my own environment, which I investigated and fixed -- not force-passed."
- "This all runs through one command and real CI on GitHub, using entirely free tooling."

5 things NOT to claim

- Do NOT say "fully verified" or "100% verified" -- the project explicitly disclaims this.
- Do NOT claim SystemVerilog Assertions or UVM experience -- this project used none.
- Do NOT claim the LLM testbench (6/6) is comparable to the hand-written suite (9/9).
- Do NOT claim formal verification covers the whole DUT -- it's 3 narrow properties on 1 of 2 DUTs.
- Do NOT claim CI was "probably fine" if you haven't actually looked at a real run -- point at the Actions tab.

26. Interview Cheat Sheet

Final rapid-revision page -- read this 10 minutes before, not the whole document.

Architecture, one line

stimulus → monitor → reference model → scoreboard + protocol/latency checkers → coverage → mutation testing → formal verification → regression automation + CI.

5 most important numbers

- 24/24 tests, 9 suites
- 9/9 mutation score on both DUTs (UART + APB), each with a different per-mechanism spread
- 3/3 formal properties PASS -- P2 and P3 unbounded (k-induction), P1 bounded-but-complete (BMC depth 5)
- 100% functional coverage on every owned coverage point/cross
- LLM experiment: v1 5/6, v2 6/6 (not comparable to the 9/9 main suite)

5 strongest technical points

- Genuinely separated mechanisms -- proven by a per-mechanism mutation-attribution table, not a claim
- Two unbounded formal proofs (k-induction), with the bounded/unbounded distinction explained correctly
- Mutation testing found and fixed two real gaps (mutant3 investigated, not force-fixed)
- Reproducibility verified byte-for-byte, not assumed
- Real, observed green CI run on GitHub Actions

5 weakest points (own them, then pivot)

- No SystemVerilog Assertions / UVM experience
- Small, deliberately simple DUTs
- No commercial EDA tool experience
- No clock-domain-crossing verification (both DUTs are single-clock)
- LLM experiment is a small, non-comparable side track

10 keywords/concepts I must understand

#	Concept
1	Constrained-random verification
2	Functional coverage (coverpoints, crosses, bins)
3	Scoreboard vs. protocol checker vs. latency checker
4	Mutation testing (CAUGHT/SURVIVED, equivalent mutants)
5	BMC vs. k-induction
6	APB SETUP/ACCESS handshake
7	UART 8N1 framing

#	Concept
8	Reference model independence
9	Seed-based reproducibility
10	Separation of concerns in verification architecture

10 questions I should be able to answer cold

- What does a mutation score of 100% actually mean, and not mean?
- Why is k-induction unbounded but BMC (usually) isn't?
- Why is P1 (BMC) still called a complete proof?
- Why does register-map legality live in the scoreboard, not the protocol checker?
- Why doesn't the scoreboard check APB's OP_LATENCY?
- Walk me through the mutant3 investigation, start to finish.
- Why didn't you use SVA?
- How do you reproduce a random-test failure?
- What's the difference between what mutation testing and formal verification each prove?
- Why is 6/6 on the LLM experiment not comparable to 9/9?

27. "If They Ask Me to Go Deeper"

Three progressively deeper levels for each major component -- stop at whichever level answers the question.

cocotb architecture

Level 1 (simple): cocotb lets you write a testbench in Python that drives and observes a Verilog/VHDL DUT through a simulator.

Level 2 (technical): Each `@cocotb.test()` is an async coroutine; `await RisingEdge(dut.clk)` and similar triggers yield control back to the simulator's event loop until that event occurs -- so multiple concurrent tasks (driver, monitor, checkers) run interleaved, cycle-accurately.

Level 3 (implementation): Icarus Verilog is driven via cocotb's VPI interface. The Makefile-based flow (Makefile.icarus) manages compilation and simulation invocation; `MODULE` selects which Python file's tests run, `VERILOG_SOURCES` selects which RTL file compiles -- which is exactly the mechanism the mutation engine exploits (swap `VERILOG_SOURCES`, keep `MODULE` fixed) to test the same testbench against different RTL.

scoreboard

Level 1 (simple): It's the referee -- compares what the DUT actually did against what it should have done.

Level 2 (technical): Monitor-observed transactions are compared against a reference model's predictions, transaction by transaction, using `expect()/check()` calls that raise on mismatch. It knows nothing about timing.

Level 3 (implementation): UART's scoreboard compares monitor-decoded byte + per-segment samples against `reference_model.expected_frame_bits`; APB's scoreboard additionally checks register-map legality (whether `PSLVERR` should have been raised) by consulting the same reference model's address-map logic, not a separate lookup table.

reference model

Level 1 (simple): An independent, hand-written prediction of what the DUT should output, given the same inputs.

Level 2 (technical): Pure Python, no cocotb/DUT dependency -- driven by the exact same sequence of driver calls the DUT receives, so it can predict expected output transaction by transaction.

Level 3 (implementation): APB's reference model is a small state machine (`ENABLE/DATA/BUSY/DONE`) that's deliberately 'software-realistic' -- it only learns an operation finished by being told `BUSY=0` was polled and observed, exactly mirroring what real firmware would know, which is precisely why it cannot independently verify exact `OP_LATENCY` (that's the latency checker's separate job).

coverage

Level 1 (simple): Did we exercise the important scenarios, not just line-execute the RTL.

Level 2 (technical): cocotb-coverage's CoverPoint decorator samples a value on every relevant transaction and buckets it into named bins; a Cross combines two CoverPoints to require every combination be hit.

Level 3 (implementation): `coverage_db` is a process-wide singleton -- importing `uart_env.coverage` registers every CoverPoint defined anywhere in that module for the current simulation process, regardless of which test module actually samples them, which is why each module's own report must report only its owned bins' percentage, not the process-wide aggregate (confirmed and documented, not assumed).

mutation testing

Level 1 (simple): Break the RTL on purpose, on paper, and see if the tests notice.

Level 2 (technical): `mutation/framework.py`'s `run_suite()` drives `make clean && make VERILOG_SOURCES=MODULE=` for every (mutant, mechanism) pair and parses cocotb's own `FAIL=` summary line to classify `CAUGHT`

(FAIL>0) vs SURVIVED (FAIL==0).

Level 3 (implementation): Each DUT's `mutation_suite.py` is a thin manifest of `Mutant/Mechanism` dataclasses built on the shared, unmodified engine -- meaning a future 3rd DUT needs only a new manifest, zero new runner code. Equivalent mutants get an `equivalent=True` flag, excluded from scoring but still documented with a reason.

formal

Level 1 (simple): A solver checks a claim against every possible scenario at once, instead of one scenario at a time.

Level 2 (technical): `SymbiYosys` (`sby`) orchestrates `Yosys` (RTL-to-logic frontend) and `yosys-smtbmc` (drives an SMT solver, here `Z3`) through either `bmc` mode (fixed-depth unrolling) or `prove` mode (`k`-induction: base case + induction step).

Level 3 (implementation): This toolchain doesn't reliably model hierarchical cross-module references (confirmed via a trivial tautology producing a false counterexample), so both `P2` and `P3` observe the DUT exclusively through real output ports (`busy_o`, `data_o` added specifically for this) rather than `dut.busy_reg`-style references -- a targeted workaround for a verified tool limitation, not a design preference.

APB

Level 1 (simple): A simple two-phase bus protocol: address/data get set up, then the transfer happens.

Level 2 (technical): `SETUP` (`PSEL=1`, `PENABLE=0`) then `ACCESS` (`PSEL=1`, `PENABLE=1`) the next cycle; `PREADY` (tied high here) lets a slave insert wait states by staying low during `ACCESS`, which this DUT never needs since it's zero-wait-state.

Level 3 (implementation): The `START/ENABLE` control logic checks the registered `enable_reg`, not the incoming `PWDATA` bit in the same write -- a same-cycle write of both `ENABLE` and `START` does not start an operation, only a `START` arriving after `ENABLE` was already committed does. This exact control-logic subtlety is what `mutant3`'s busy-guard removal broke, and what formal property `P2` now proves exhaustively.

UART

Level 1 (simple): Start bit, 8 data bits LSB-first, stop bit, each held for a fixed number of clock cycles.

Level 2 (technical): A small FSM (`IDLE/START/8x DATA/STOP`) driven by a `clk_count` counter that increments each cycle within a bit period and resets when it reaches `CLKS_PER_BIT-1`, advancing the FSM state on that boundary.

Level 3 (implementation): The protocol checker samples `tx_serial` every single clock cycle of every segment (not at a fixed mid-bit offset) specifically because a one-cycle-short or one-cycle-long segment is exactly the defect class two of the nine mutants (`mutant3`, `mutant9`) model, and a fixed-offset sample -- as the `phase2_llm v1` testbench demonstrated -- can miss it entirely.

regression framework

Level 1 (simple): One command runs everything and gives you one report.

Level 2 (technical): `run_regression.py` sequences 9 make-based test suites, lint, both mutation suites, and 3 formal proofs, using `regression/runner.py` and `regression/formal.py` as pure subprocess-driving orchestrators -- it never imports `uart_env/apb_env` code directly.

Level 3 (implementation): Coverage extraction happens immediately after each suite runs (not in a batched final pass) because every suite's make clean deletes the DUT's shared `sim_build/` directory -- an early version of this orchestrator got this wrong and produced an empty coverage section, a real bug found and documented, not a design choice made correctly the first time.

28. Final Pre-Interview Checklist

Run through this 10 minutes before the call or demo.

- [] Repository opens (github.com/sanjanaa2102/vlsi-verification-project) and README renders correctly
- [] python run_regression.py runs cleanly from the repo root (or a recent report.md is on hand as backup)
- [] Commands memorized or in a visible cheat sheet: mutation_suite.py, WAVES=1 + gtkwave, --skip-mutation --skip-lint
- [] Architecture understood well enough to draw the stimulus-to-CI chain from memory (Section 3)
- [] Mutation example ready: APB mutant3 diff open or screenshotted, story rehearsed (Section 7)
- [] Waveform ready: sim_build/apb_regblock.fst generated, GTKWave tested and opens without error
- [] Formal result understood: can explain BMC vs. k-induction and why P1 is 'bounded but complete' without notes
- [] LLM explanation ready: 5/6 -> 6/6 story and why it's not comparable to 9/9 (Section 17)
- [] Resume bullets understood well enough to expand any one of them on request
- [] Weaknesses prepared: all 5 from Section 22, in the 'name it, then pivot' order

End of guide. Source of truth for every fact above: README.md, ARCHITECTURE.md, VERIFICATION_STRATEGY.md, LESSONS_LEARNED.md, phase1_uart/VERIFICATION_PLAN.md, phase5_apb/VERIFICATION_PLAN.md, and directly re-measured phase2_llm mutation runs, all in this repository.